Chapitre 8- Les structures de données avancées

I Introduction

Notion d'abstraction :

Une structure de donnée abstraite est un **type** permettant une organisation structurée d'informations élémentaires.

On manipule les différentes informations à travers des opérations prédéfinies sans avoir nécessairement connaissance de leur forme interne.

Etudier un **type abstrait** revient :

- à définir d'abord ce que l'on peut faire sur ce type
- ensuite comment il convient de le faire en fonction de la forme interne donnée au type.

Exemple (de structure de donnée déjà vues) :

- tables
- structures
- fichiers

Nous allons voir de nouvelles structures de données telles que les listes linéaires, les piles, les files, les arbres.

Elles seront présentées à l'aide de types abstraits avec une ***définition externe*** et une ***définition interne***.

Chapitre 2- Les structures de données

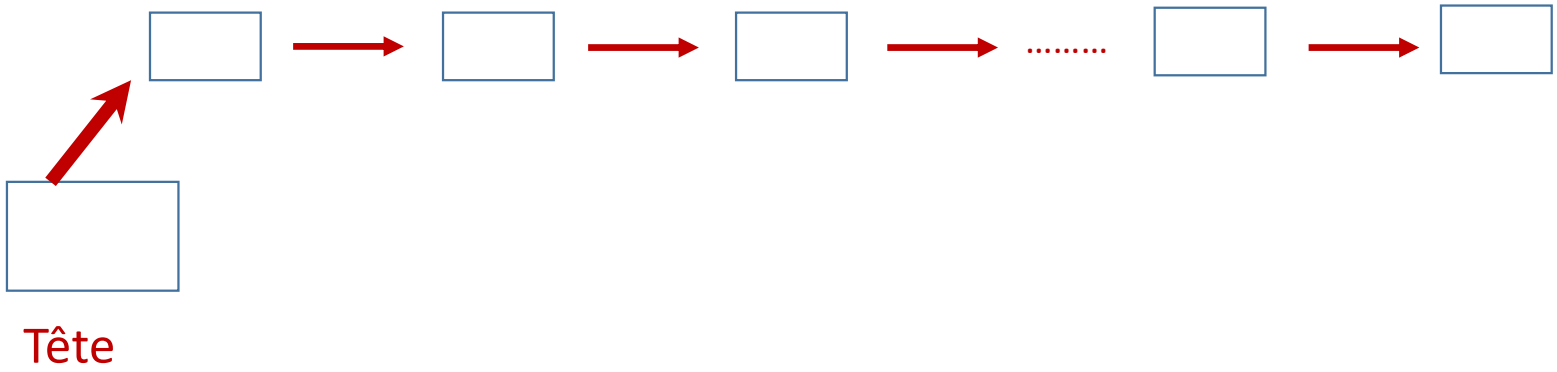
II Définition externe des structures de données (SD) les plus courantes

II.1 Définition d'une liste linéaire

- C'est un ensemble fini et ordonné d'éléments de même type
- Tout élément connaît son suivant (son **successeur**) dans la liste
- Le dernier élément n'a pas de successeur
- Il existe un élément spécifique, la tête de liste, qui permet d'accéder au premier élément de la liste
- Les ajouts et les suppressions peuvent s'effectuer à n'importe quel endroit de la liste

Chapitre 2- Les structures de données

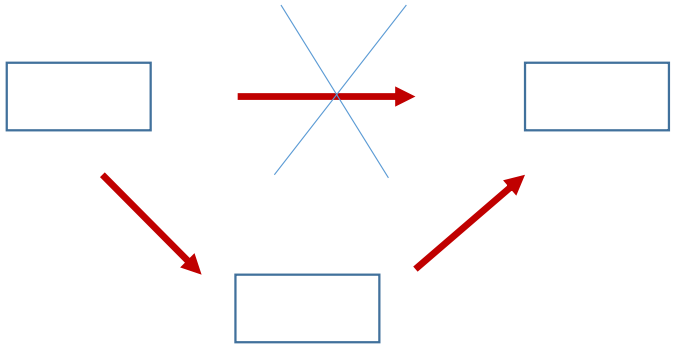
Schématisation :



- ✓ **Ajouter un élément :** casser le lien entre deux éléments, en ajouter un et reconstruire les liens
- ✓ **Supprimer un élément :** casser deux liens pour supprimer l'élément au milieu, reconstruire le lien entre le précédent et le suivant

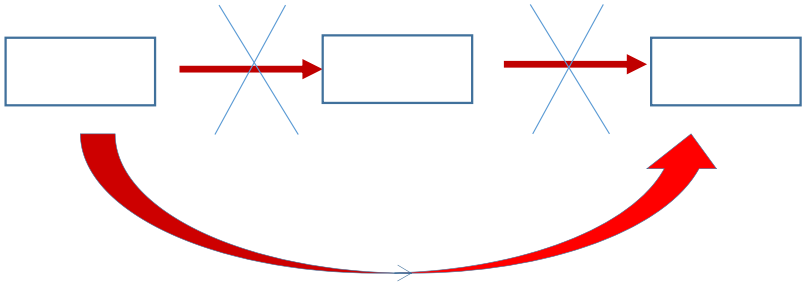
Chapitre 2- Les structures de données

Ajout



Casser des liens et en recréer

Suppression



Casser des liens et en recréer

Chapitre 2- Les structures de données

Définition externe

- Soit T_Elt le type des éléments de la liste.

- Soit T_Liste le type ˝liste linéaire˝ qui contient un ensemble de données de type T_Elt

Chapitre 2- Les structures de données

Liste d'opérations :

(NB : Pour simplifier, on adopte une écriture algorithmique)

- *Création :*

fonction Creation **:** T_Liste;

-- renvoie une liste vide (avec seulement l'entête)

- *Ajout d'élément :*

module Ajout_En_Tete (↕ L : T_Liste; E : T_Elt)

module Ajout_En_Queue (↕ L : T_Liste; E : T_Elt)

Chapitre 2- Les structures de données

- *Manipulation :*

fonction Premier (L : T_Liste) : T_Elt

-- *renvoie la valeur de l'élément en tête*

fonction Suivant (L : T_Liste) : T_Liste

-- *renvoie la liste sans le premier élément.* ***C'est une fonction de parcours.***

fonction Liste_Vide (L : T_Liste) : Booléen

Chapitre 2- Les structures de données

- *Suppression :*

module Enleve_En_Tete (↕ L : T_liste)

-- supprime le premier élément

module Enleve_En_Queue (↕ L : T_Liste)

-- supprime le dernier élément

Enleve_En_Tete ≠ Suivant

(modifie la liste) *(ne modifie pas la liste)*

Chapitre 2- Les structures de données

- *Autres :*

module Detruire (↕ L : T_Liste)

-- détruit tous les éléments de la liste

fonction Liste_Pleine (L : T_Liste) : Booléen

-- dépend de la définition interne de la liste (son implémentation)

La définition que l'on vient de voir décrit la forme générale de structures de données linéaires.

Il existe des formes plus restreintes de ce type : *la pile, la file*.

Chapitre 2- Les structures de données

II.2 La pile

- C'est un cas particulier des listes linéaires.
- Les insertions/suppressions peuvent se faire :
  - ✓soit en tête de pile
  - ✓soit en queue de pile (XOR disjonction exclusive : à vrai si les valeurs des opérandes sont différentes)

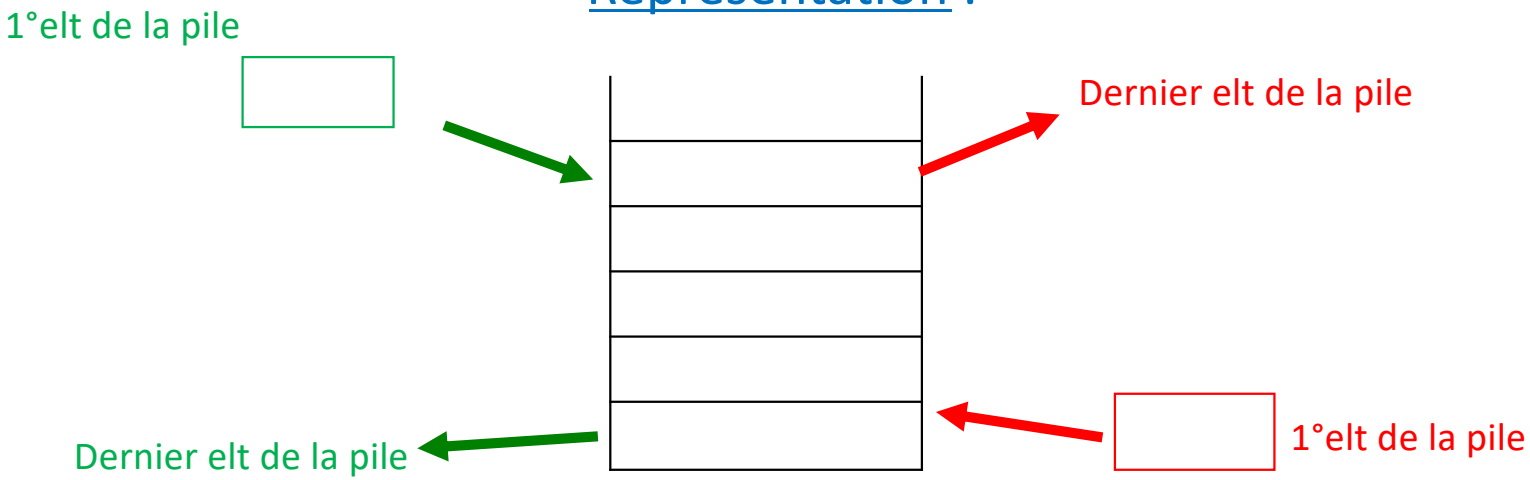
C'est une structure LIFO : Last In First Out (dernier entré, premier sorti).

(*ou encore FILO : First IN Last Out)*

Rappel : *Le principe de la pile est utilisé dans la gestion des exécutions des sous-programmes : on empile l'environnement d'exécution à l'appel, on dépile l'environnement à la fin de l'appel.*

Chapitre 2- Les structures de données

Représentation :



C'est bien la représentation d'une liste linéaire sous forme de pile.

- Dépiler ⇒ enlever en queue
- Empiler ⇒ ajouter en queue

On peut aussi regarder dans l'autre sens :

- Dépiler ⇒ enlever en tête
- Empiler ⇒ ajout en tête

Chapitre 2- Les structures de données

Définition externe

- Soit T_Elt le type des éléments de la pile.

- Soit T_Pile le type de la pile qui contient un ensemble d'éléments de type T_Elt

Chapitre 2- Les structures de données

Liste d'opérations :

fonction Creation **:** T_Pile

-- création d'une pile vide

module Empiler (↕ P : T_Pile; E : T_Elt)

-- au sommet de la pile

module Depiler (↕ P : T_Pile)

-- depuis le sommet de la pile

fonction Sommet (P : T_Pile) **:** T_Elt

-- valeur de l'élément

fonction Pile_Vide (P : T_Pile) : Booleen

module Detruire(↕ P : T_Pile)

Chapitre 2- Les structures de données

II.3 La file

- C'est également un cas particulier des listes linéaires.
- Les insertions/suppressions se font :
  - ✓ soit en queue de file (insertion)
  - ✓ soit en tête de file (suppression)

C'est une structure **LILO** : Last In Last Out (dernier entré, dernier sorti).

(ou encore FIFO First In First Out)

Remarque : Le principe de la file est utilisé dans la gestion des processus qui veulent accéder aux ressources d'un ordinateur lorsqu'on ne dispose que d'un seul processeur par exemple.

Chapitre 2- Les structures de données

- Soit T_Elt le type des éléments de la file.

- Soit T_File le type de la file qui contient un ensemble de valeurs de type T_Elt

Chapitre 2- Les structures de données

Liste d'opérations :

fonction CREATION_FILE : T_FILE

-- création d'une file vide

fonction FILE_VIDE (F : T_FILE) : Booléen

-- teste si une file est vide

fonction TETE (F : T_FILE) : T_ELT

-- valeur de l'élément en tête

module ENFILER (↕ F : T_FILE; E : T_ELT)

-- ajouter en queue

module DEFILER (↕ F : T_FILE)

-- supprimer en tête

module DETRUIRE (↕ F : T_FILE)

-- destruction de la file

fonction SUIVANT (F : T_FILE) : T_FILE

-- renvoie le suivant qui est une file

fonction QUEUE (F : T_FILE) : T_FILE

-- renvoie le dernier qui est une file